Project 3: Random Matrices

Reyhaneh Bahranifard

April 24, 2026

Contents

1 Introduction **3**

2 Methods **3**
2.1 Manager-Worker Model 3
2.2 Power Iteration Algorithm 5
2.3 Schur Decomposition 6
2.4 Kolmogorov-Smirnov Test 7

3 Results **7**
3.1 Parallel Optimization 7
3.2 Eigenvalues Distribution 9

4 Conclusion **12**

1 Introduction

Random matrices emerged for the first time when Eugene P. Wigner tried to describe very dense nuclear energy levels. He showed that in a long-range interaction, the width and spacing of nuclear resonance levels can be treated statistically since they have universal behavior totally independent of the detailed physics behind them.[1] Four years later, he tried to model this assumption mathematically. He modeled complex quantum systems using large random matrices, considered eigenvalues as the energy levels and eigenvectors as quantum states. He defined random matrices as matrices whose entries were chosen randomly from a probability distribution and are real and symmetric. He showed that most eigenvalues lie in a finite range and the density forms a semicircle distribution.[2] Later, Tracy and Widom studied the statistical behavior of eigenvalues of large random hermitian matrices, specifically focusing on the edges of this semicircle distribution where the largest eigenvalues sit. They derived an exact mathematical expression for the probability distribution of the largest eigenvalue when $N \to \infty$ [3], which is nowadays known as the "Tracy-Widom distribution". This distribution looks like a Gaussian but it is not symmetric at the tails.
In this project, we tried to reproduce the Tracy-Widom distribution for the largest eigenvalues of large random matrices and compare it to a Gaussian. We considered a symmetric matrix A where the off-diagonal elements of A are drawn from the standard normal distribution $N(0, 1)$ and the diagonal elements are drawn from the normal distribution $N(0, 2)$. Since the difference is only at the tails, which is where rare events sit, we need to produce a large number of random matrices and calculate their largest eigenvalue to be able to see this subtle difference. Producing large matrices and calculating the largest eigenvalue, which involves a large amount of matrix multiplication, is not only an expensive calculation, but also requires a lot of memory space. To make this calculation feasible, the program was parallelized using MPI, where different copies of the code with their own memory space are produced to solve the memory constraint.

2 Methods

2.1 Manager-Worker Model

The manager-worker model is a parallelization method in which one process, the manager, assigns tasks to other processes, the workers. The manager does not have heavy computational duties itself but keeps workers busy at all times. It collects results and decides when enough work has been done. As a result, workers do not need to wait for slower workers; they can work on other tasks as soon as they are done with their own. In the following, the duties of the manager and workers are discussed explicitly for this project.

Algorithm 1 Manager Process

```
Input: matrix size n, ensemble size ndat
Output: a file including ndat largest eigenvalues
call init(n, ndat)
mpi_bcast(n) to all processes                          ▷ share matrix size with all workers
allocate eigs(ndat)
start(1:num_procs−1) ← True                            ▷ flag: is worker ready for new seed?
exit_tags_sent ← 0                                     ▷ number of workers sent EXIT signal
failed ← 0                                             ▷ number of unconverged eigenvalues
recvd ← 0                                              ▷ number of eigenvalues received so far
open file 'eigs'
while True do
    for p = 1 to num_procs−1 do                        ▷ loop over all workers
        if start(p) == True then                       ▷ worker p is ready for new work
            generate random seed                       ▷ unique seed for each matrix
            if recvd ≤ ndat − num_procs + 2 then       ▷ still need more eigenvalues
                tag ← DEFAULT                          ▷ signal worker to compute eigenvalue
            else
                tag ← EXIT                             ▷ signal worker to exit
                exit_tags_sent ← exit_tags_sent +1
            end if
            mpi_send(seed, tag, to worker p)           ▷ send work to worker
            start(p) ← False                           ▷ mark worker as busy
        end if
    end for
    if exit_tags_sent == num_procs−1 then              ▷ all workers stop
        exit while loop
    end if
    mpi_recv(eigenvalue, from ANY worker, ANY tag)     ▷ wait for any result
    worker ← status(MPI_SOURCE)                        ▷ which worker sent this result
    tag ← status(MPI_TAG)                              ▷ 0 = converged, 1 = failed
    if tag == 0 then                                   ▷ eigenvalue computation converged
        recvd ← recvd +1
        eigs(recvd) ← eigenvalue
        write worker, eigenvalue to file
    else                                               ▷ power iteration failed to converge
        failed ← failed +1
    end if
    start(worker) ← True                               ▷ mark worker as ready for new work
end while
close file
```

The manager generates random seeds and sends them to workers, receives eigenvalues back, writes them to the output file, and decides when to stop by sending an $EXIT_TAG$ to workers. On the other hand, when workers receive a seed, they start the calculation and return the largest eigenvalue before receiving another seed. Two approaches were used to calculate the largest eigenvalue. In the case that both fail, the worker returns a fail tag to the manager. The process continues until the manager sends the $EXIT_TAG$ to all workers. Manager and worker pseudo-codes are provided as Algorithm 1 and Algorithm 2, respectively.

Algorithm 2 Worker Process

```
Input: matrix size n (from manager), random seed (from manager)
Output: largest eigenvalue sent back to manager
mpi_bcast(n) from manager                                   ▷ receive matrix size
allocate matrix A(n, n)                                     ▷ give each worker one matrix
while True do
    mpi_recv(seed, tag, from manager)                       ▷ wait for work order
    if tag == EXIT then                                     ▷ manager signals to exit
        exit while loop
    end if
    open PRNG stream with seed
    make random matrix A
    compute largest eigenvalue of A, check convergence      ▷ Using power iteration or DGEEV
    if converged then                                       ▷ power iteration succeeded
        tag ← 0
    else                                                    ▷ power iteration and DGEEV both failed
        tag ← 1
    end if
    mpi_send(eigenvalue, tag, to manager)                   ▷ return result
end while
deallocate A
```

2.2 Power Iteration Algorithm

Assume matrix A has eigenvectors

$$u_1, u_2, u_3, \ldots$$

with corresponding eigenvalues

$$\lambda_1, \lambda_2, \lambda_3, \ldots$$

where

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|.$$

By multiplying a random vector $v = c_1u_1 + c_2u_2 + c_3u_3 + \cdots + c_nu_n$ by A k times, we get:

$$A^k v = c_1\lambda_1^k u_1 + c_2\lambda_2^k u_2 + \cdots + c_n\lambda_n^k u_n.$$

Factoring out λ_1^k, we get:

$$A^k v = \lambda_1^k \left(c_1u_1 + c_2 \left(\frac{\lambda_2}{\lambda_1}\right)^k u_2 + \cdots + c_n \left(\frac{\lambda_n}{\lambda_1}\right)^k u_n \right).$$

As k goes to infinity, terms involving $u_2, u_3, \ldots$ vanish since $\frac{\lambda_n}{\lambda_{n-1}} < 1$, meaning the vector converges to the eigenvector corresponding to the largest eigenvalue.[4] According to this algorithm, when we multiply a random vector by matrix A, it stretches the vector in many directions corresponding to its eigenvectors, but the direction corresponding to the largest eigenvalue magnitude gets stretched the most.

Algorithm 3 Power Iteration

```
Input: matrix A(n, n), shift μ, convergence criterion ε, max iterations M
Output: largest eigenvalue λ, convergence flag conv
conv ← False
ε ← 10^-5                                   ▷ relative convergence criterion
M ← n/2
produce a random vector v(n) from N(0, 1)
v ← v/‖v‖                                   ▷ normalize initial vector
mem ← −100                                  ▷ stores previous eigenvalue estimate
for i = 1 to M do
    y ← A · v + μv                          ▷ matrix-vector multiplication
    λ ← v · y − μ
    err ← |mem − λ|/|mem|                   ▷ relative change from previous iteration
    if err < ε then
        conv ← True
        exit loop                           ▷ convergence achieved
    end if
    v ← y/‖y‖                               ▷ normalize for next iteration
    mem ← λ                                 ▷ store current estimate
end for
if conv == True and λ < 0 then              ▷ converged to wrong eigenvalue
    μ ← −λ                                  ▷ shift to make largest eigenvalue dominant
    repeat power iteration with new μ
end if
if conv == False then                       ▷ power iteration failed
    compute full spectrum via LAPACK DGEEV  ▷ Schur decomposition fallback
    λ ← max(real parts of all eigenvalues)
    conv ← True
end if
```

Since A is symmetric, for every positive eigenvalue, there is a negative eigenvalue with equal magnitude. This means that the power iteration may converge to the largest negative eigenvalue rather than the largest positive one. To handle this, the code first runs power iteration as discussed. If the result is negative, it applies a shift $\mu = -\lambda$. Therefore, the matrix is transformed to $A + \mu I$, which shifts all eigenvalues by μ and makes the largest positive eigenvalue clearly dominant. Algorithm 3 provides a clear explanation of how power iteration was used in this project. At each time step, the estimated eigenvalue is compared with the estimate from the previous iteration; if the change is smaller than 10^{-5}, the loop ends and a True flag is returned indicating convergence. If the loop continues for the maximum number of iterations without convergence, the run is flagged as a failure.

2.3 Schur Decomposition

As discussed, power iteration does not always converge. In such cases the code falls back to LAPACK's `DGEEV`, which computes the full spectrum of the matrix via Schur decomposition. The largest eigenvalue is then selected from the complete list of eigenvalues:

$$\lambda_{\max} = \max_i \operatorname{Re}(\lambda_i), \quad i = 1, \ldots, n$$

Algorithm 4 Schur Decomposition Fallback

```
if conv == False then                                  ▷ power iteration failed to converge
    call DGEEV(A)                                      ▷ compute full spectrum using LAPACK
    if DGEEV succeeded then
        λ ← max(Re(λ_1, λ_2, ..., λ_n))                ▷ select largest eigenvalue
        conv ← True
    else
        print error
    end if
end if
```

The pseudo-code for the Schur decomposition fallback is provided in Algorithm 4. Although this method is guaranteed to converge, it is significantly more expensive than power iteration, scaling as $\mathcal{O}(n^3)$ compared to $\mathcal{O}(n^2)$ per iteration for power iteration, and is therefore used only as the last resort.

2.4 Kolmogorov-Smirnov Test

The Kolmogorov-Smirnov test is a measure of how well a sample dataset is aligned with a theoretical distribution.

$$D = |F_S(x) - F_T(x)| \tag{1}$$

Assume $F_T(x)$ is the desired theoretical cumulative distribution function and $F_S(x)$ is the cumulative distribution function of the sample data. According to Equation (1), D is the greatest distance between $F_S(x)$ and $F_T(x)$. The smaller D is, the closer the sample data is to that specific distribution. In this project, a built-in function from the scipy library was used to check whether the distribution of the largest eigenvalues follows Tracy-Widom or not. `D, p-value = scipy.stats.ks(`$F_S(x)$`,`$F_T(x)$`)`, where the p-value is the probability of observing a difference D as large or larger if the data truly came from the reference distribution. Therefore, a low p-value means the data is very unlikely to have come from the reference distribution.

3 Results

3.1 Parallel Optimization

Wall time is the actual elapsed time from the start of a program to its end. In this project, wall time is proportional to the number of required eigenvalues (N), the number of processes (P), and the matrix size n, which appears in the power iteration algorithm as $\mathcal{O}(n^2)$ and in the DGEEV fallback as $\mathcal{O}(n^3)$. There is also a time contribution from MPI communication overhead, which includes sending seeds and receiving results, and is completely independent of n or N. The code was run for each of these values and the results were written to .txt files, which are provided on the GitHub repository as speed_ndat{N}.txt. Since workers split the eigenvalue calculations, wall time can be expressed as:

$$\text{wall time} \propto \frac{N}{P} \times t_n + t_{MPI} \tag{2}$$

According to this relation, there are two terms each competing to dominate the wall time. To investigate the parallel performance of the implementation, the wall time and the speedup graph as a function of the number of MPI processes are provided in Figure 1 and Figure 2, respectively, once with a fixed number of required eigenvalues and once with a fixed matrix size.

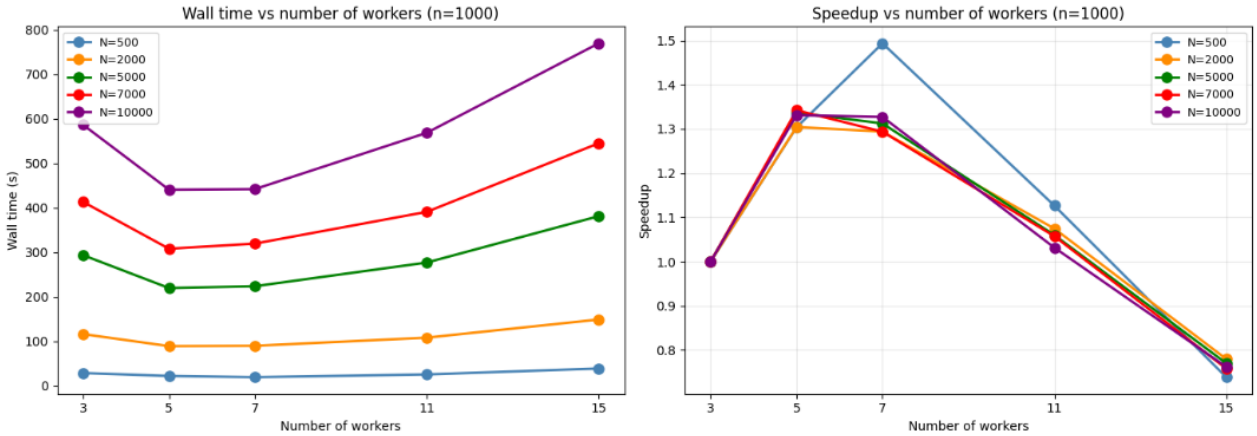

Figure 1: Wall time vs. number of workers and the corresponding speedup graph. All runs were performed on a single machine with 16 processes. The number of calculated eigenvalues is $N = [500, 2000, 5000, 7000, 10000]$ with matrix size fixed at 1000×1000. There is a universal optimum point around 5 or 7 workers for all graphs. The speedup graph was normalized with respect to the 3-worker data.

As shown in Figure 1, increasing N generally increases the wall time. Since increasing P eventually makes t_{MPI} dominant, increasing the number of processes is not always a solution to optimize the code. Doubling N scales the wall time up but does not change the shape of the speedup graph. This shows that regardless of P, the manager process and MPI communication overhead are a serious limitation, and as discussed, they are independent of n or N.

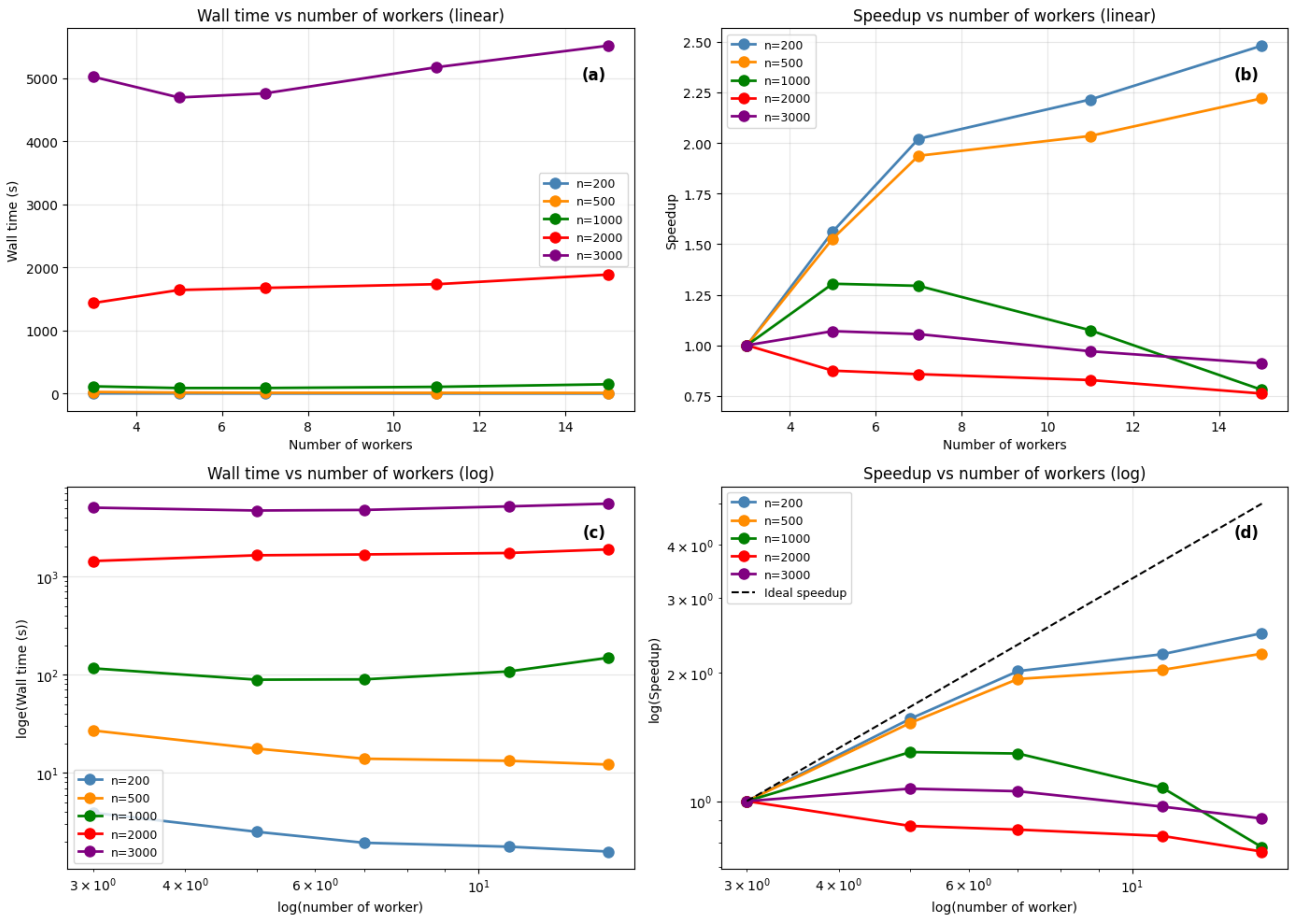

Figure 2: (a) and (b) Wall time vs. number of workers and the corresponding speedup graph with fixed $N = 2000$ for various matrix sizes $n = [200, 500, 1000, 2000, 3000]$. (c) and (d) Log-log plots of (a) and (b).

According to Equation 2, wall time is proportional to N and to n^3 or n^2. Therefore, wall time is more sensitive to matrix size than to N. Looking at Figures 2(a) and 2(b), more dramatic changes are observed when changing the matrix size n compared to changing N (Figure 1); for example, $n = 3000$ takes roughly 50 times longer than $n = 1000$.
Log-log plots of wall time vs. number of workers are provided in Figure 2(c) and 2(d) for comparison. Small matrices benefit from parallelization because computation is fast and workers stay busy. Large matrices show poor scaling because each matrix computation is so expensive that the manager becomes the bottleneck; it cannot distribute seeds and collect results fast enough to keep 15 workers busy simultaneously. In other words, the MPI communication overhead dominates Equation (2). From this point on, 7 workers were used for all subsequent runs.

3.2 Eigenvalues Distribution

The code was run for different matrix sizes and different N, and the eigenvalues were saved and provided in the GitHub repository for each run.

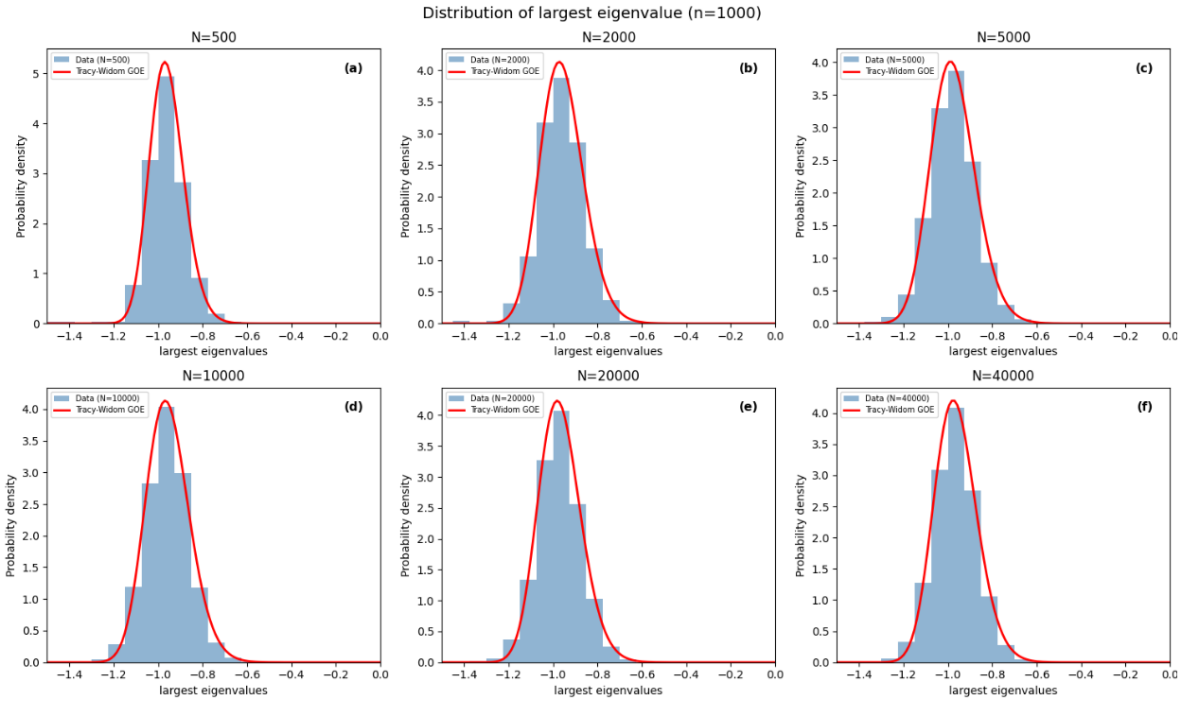

Figure 3: Sample distribution for a fixed matrix size of 1000 with different N. As N increases, the histogram becomes smoother and fills the tails of the fitted Tracy-Widom distribution better. No data has been removed. Graphs are zoomed in using `plt.xlim` to make the main peak clearer.

In Figure 3, the distribution of data for a fixed matrix size ($n = 1000$) for different values of N is provided. In this graph, the eigenvalues were first standardized by subtracting the sample mean and dividing by the sample standard deviation, then rescaled to match the known mean and standard deviation of the standard Tracy-Widom GOE distribution (-1.2065 and 1.2680 respectively). The `curve_fit` function from scipy was then used to find the best location and scale parameters that minimized the difference between the Tracy-Widom PDF and the histogram of the standardized data. The fitting was applied only to the main peak of the distribution to avoid the influence of outliers from wrong power iteration convergences. As shown in this figure, as the number of calculated eigenvalues (N) increases, the sample histogram becomes smoother and more visible. However, even with 40000 samples, the sample histogram does not fill the area under the fitted curve, especially on the right tail where very rare events sit. This argument is even more evident in Figure 4, where all the samples are plotted together.

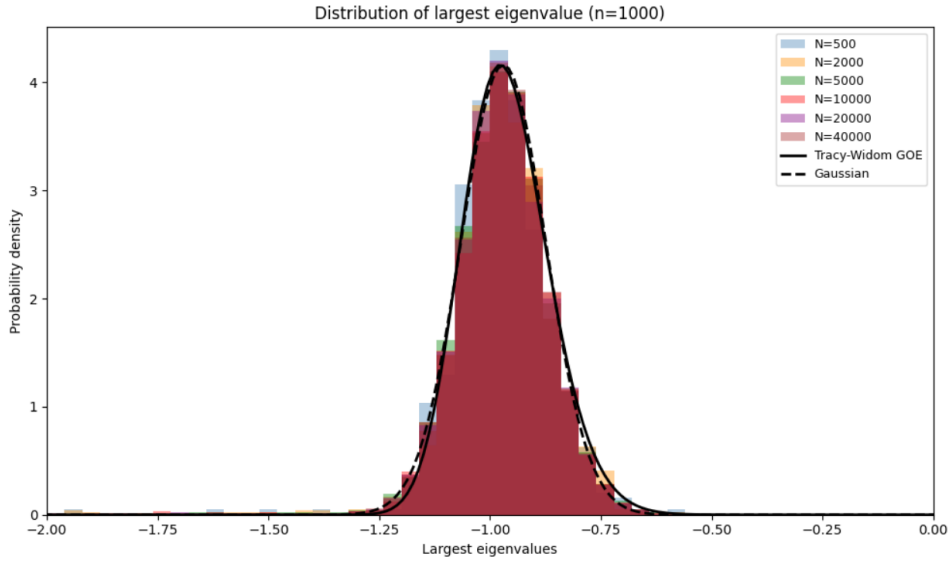

Figure 4: Distribution of the largest eigenvalue for all collected samples with matrix size 1000. Different colors represent different sample sizes. Noisy spikes in the graph are associated with smaller sample sizes, showing that larger sample sizes produce smoother distributions. Outliers from wrong convergences are visible at small eigenvalue values.

According to Figure 4, the Tracy-Widom fit and the Gaussian fit appear almost identical in the bulk, but the difference is in the tails where rare events sit. The left tail of the data is well aligned with the Tracy-Widom distribution, but the right tail is not perfectly matched since the matrix size is not large enough to populate those rare events.

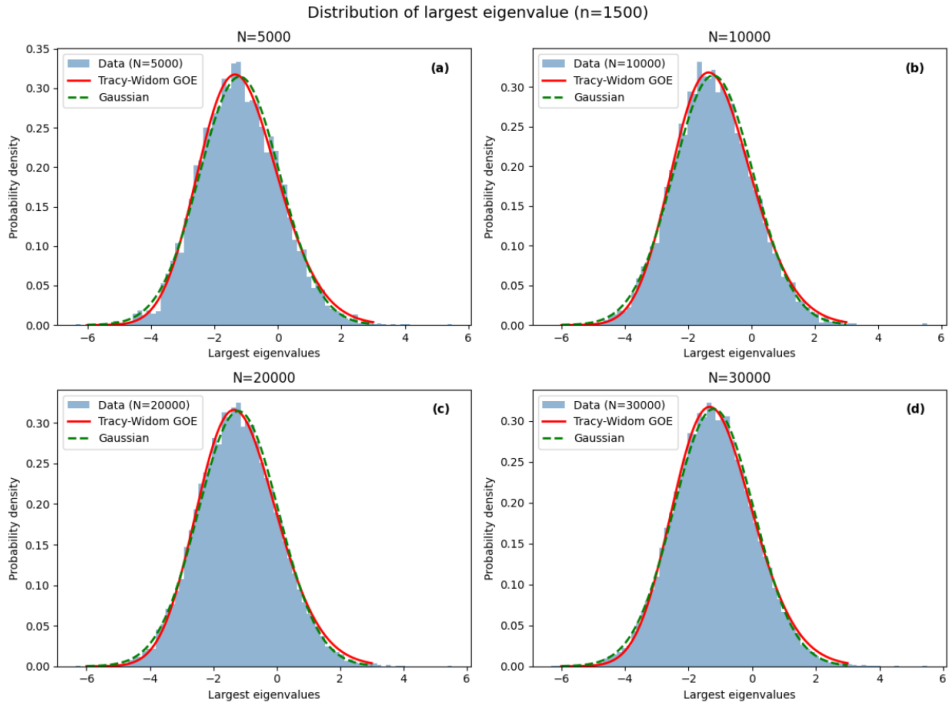

Figure 5: Sample distribution for a fixed matrix size of 1500 with different N. To keep the wall time feasible, $N = 5000$ was run 4 times and the results were combined for each graph.

In Figure 5, the same test was performed for $n = 1500$. For $n = 1000$, many outliers were produced far from the main peak. These outliers inflate the standard deviation, and since the data is standardized by dividing by this value, the main peak is compressed into a narrow spike. For $n = 1500$, almost no outliers are present, so the standard deviation reflects only the true width of the distribution and the histogram appears with its natural shape. Therefore, distributions in Figure 4 appear narrower than those in Figure 5.
The KS statistic measures the maximum difference between the sample cumulative distribution function and the Tracy-Widom theoretical distribution.

Table 1: KS statistic between the sample eigenvalue distribution and Tracy-Widom for different matrix sizes n and ensemble sizes N.

n	N	KS statistic
1000	500	0.4825
1000	2000	0.4574
1000	5000	0.4601
1000	10000	0.4589
1000	20000	0.4599
1000	40000	0.4598
1500	5000	0.0099
1500	10000	0.0090
1500	20000	0.0117
1500	30000	0.0125

As shown in Table 1, for $n = 1000$ the KS statistic remains consistently large (≈ 0.46) regardless of the ensemble size N, indicating poor agreement with the Tracy-Widom distribution. This is a direct consequence of the incorrect convergences discussed above, which distort the distribution and prevent meaningful comparison with the theoretical prediction. In contrast, for $n = 1500$ the KS statistic drops dramatically to values below 0.013, confirming that the empirical distribution is in good agreement with Tracy-Widom. Furthermore, for $n = 1500$ the KS statistic does not decrease significantly as N increases from 5000 to 30000, suggesting that the ensemble size is already sufficient to characterize the distribution and that the remaining deviation from Tracy-Widom is due to the non-standard nature of our matrix ensemble rather than insufficient sampling.
For $n = 1500$ with $N = 10000$, the KS statistic against the Tracy-Widom distribution is 0.0090, while the corresponding KS statistic against a Gaussian distribution with the same mean and standard deviation is 0.0154. The smaller KS statistic for Tracy-Widom confirms that the empirical distribution is closer to the Tracy-Widom prediction than to a Gaussian, consistent with the theoretical expectation for the largest eigenvalue of large random symmetric matrices.

4 Conclusion

In this project, the manager-worker model was successfully implemented using MPI to compute the largest eigenvalue distribution of large random symmetric matrices in parallel. The parallel scaling analysis showed that the optimal number of workers depends on the matrix size: for large matrices, the computation per worker dominates the communication overhead, while for small matrices, the MPI communication overhead limits the parallel efficiency.

The eigenvalue distribution was computed for matrix sizes $n = 1000$ and $n = 1500$ with ensemble sizes up to $N = 40000$ and $N = 30000$, respectively. For $n = 1000$, a larger number of incorrect convergences from power iteration was observed. For $n = 1500$, the incorrect convergence rate dropped significantly and the KS test confirmed that the sample distribution is statistically consistent with the Tracy-Widom distribution ($D = 0.009$) while being inconsistent with a Gaussian distribution ($D = 0.015$). This provides clear numerical evidence that the largest eigenvalue of large random symmetric matrices follows the Tracy-Widom distribution, as predicted by random matrix theory.

References

[1] E. P. Wigner. On the statistical distribution of the widths and spacings of nuclear resonance levels. *Mathematical Proceedings of the Cambridge Philosophical Society*, 47:790–798, 1951.

[2] E. P. Wigner. Characteristic vectors of bordered matrices with infinite dimensions. *Annals of Mathematics*, 62(3):548–564, 1955.

[3] Craig A. Tracy and Harold Widom. Level-spacing distributions and the Airy kernel. *Communications in Mathematical Physics*, 159(1):151–174, 1994.

[4] wikipedia. Power iteration.